Streamify

Movie Browsing and Search Application

Streamify is a React movie application that allows users to browse movies and filter them by title, genre, and rating. The application demonstrates practical React development concepts including functional components, hooks, props, event handling, conditional rendering, list rendering, API integration with Axios, loading states, and graceful error handling.

Project Summary

- Project name: Streamify
- Application type: Movie browsing and search app
- Frontend library: React
- Build tool: Vite
- HTTP client: Axios
- Movie data source: TMDb API when VITE_TMDB_API_KEY is configured, with a curated local fallback collection
- Primary users: People who want a quick way to discover and filter movies

Objective

The objective of this project is to build a simple and user-friendly movie app where users can search for films by title and narrow results using genre and rating filters. The app is designed to be clear, responsive, and reliable even when an external API is unavailable.

Key Features

- Browse a collection of movie cards with poster, title, release year, rating, genres, overview, and runtime where available.
- Search movies instantly by typing a title into the search field.
- Filter movies by genre using a dropdown generated from the available movie data.
- Filter movies by minimum rating using a range input.
- Reset all active filters with a single button.
- Show a loading indicator while movie data is being fetched.
- Display friendly messages for API fallback, errors, empty results, and retry actions.
- Use responsive layout styles so the app works on desktop and mobile screens.

React Concepts Used

- Functional components are used for App, Header, SearchFilters, MovieGrid, MovieCard, and status messages.
- useState manages movie data, loading state, errors, selected genre, search text, and selected rating.
- useEffect loads movie data when the app first renders.
- useCallback keeps the movie-loading function stable when it is passed to retry logic.

- useMemo computes the available genre list and filtered movie results efficiently.
- Props pass data and event handlers from App into child components.
- Event handling is used in search input, select dropdown, rating slider, reset button, retry button, and image error handling.
- Conditional rendering displays loading, notices, errors, empty state, poster fallback, and runtime only when needed.
- List rendering is used for movie cards and genre tags.

API Integration

The application uses Axios in src/services/movieApi.js. If VITE_TMDB_API_KEY is present in the .env file, Streamify calls the TMDb discover movie endpoint and normalizes the API response into the format used by the UI. If no key is configured, or if the API request fails, the app loads public/movies.json as a curated fallback data source.

This fallback approach keeps the user experience stable. Instead of breaking the interface when there is a network issue or an invalid API response, the app continues to show usable movie data and displays a notice explaining that the curated collection is being shown.

Component Structure

- App.jsx: Main container that loads data, stores state, computes filters, and passes props to child components.
- Header.jsx: Displays the application name, visible result count, total movie count, and active data source.
- SearchFilters.jsx: Contains the search box, genre dropdown, rating slider, and reset button.
- MovieGrid.jsx: Renders the list of movies or an empty state when no movie matches the filters.
- MovieCard.jsx: Displays movie details and handles poster image fallback when an image cannot load.
- StatusMessage.jsx: Provides reusable loading, notice, error, and retry UI.
- movieApi.js: Handles Axios requests, TMDb response normalization, fallback loading, and user-friendly API notices.

State Management

The app keeps state management simple and local because the project scope is focused. The movie list, data source label, notice message, loading flag, error message, search term, selected genre, and minimum rating are all stored in App.jsx using the useState hook. Derived values such as the genre list and filtered results are calculated with useMemo.

User Experience

- The interface begins with the actual movie browsing tool rather than a landing page.
- The filter controls are placed at the top so users can quickly refine results.
- The result count updates as filters change, giving immediate feedback.
- The app avoids blank screens by showing loading, error, fallback, and empty-result states.
- Movie cards are compact and scannable, with key details visible at a glance.
- The layout adapts for smaller screens by stacking controls and cards cleanly.

Setup and Run Steps

- Open the streamify project folder.
- Install dependencies with npm.cmd install on Windows PowerShell if npm is blocked by script policy.
- Create a .env file and add VITE_TMDB_API_KEY=your_key to use TMDb.
- Start the development server with npm.cmd run dev.
- Open the local URL shown by Vite, normally http://127.0.0.1:5173/.
- Build the production version with npm.cmd run build.

Error Handling

The app handles API problems gracefully. If TMDb returns an error status, if the request cannot reach the server, or if another request failure occurs, Streamify catches the error and loads local movie data instead. When a full data load fails, the user sees a clear error message and a retry button.

Conclusion

Streamify satisfies the project goal by providing a simple React movie app with title search, genre filtering, rating filtering, Axios API integration, loading feedback, and graceful API error handling. The project uses core React concepts in a clean component structure and remains easy to extend with additional features such as favorites, pagination, detailed movie pages, or advanced sorting.